

Внедрение Agent OS v0.8 в проект C:\AI Advisor

1. Перейти в проект

Поскольку в пути есть пробел, используйте кавычки:

```
Set-Location "C:\AI Advisor"
```

Сокращённый вариант:

```
cd "C:\AI Advisor"
```

Проверьте Git-состояние:

```
git status  
git branch --show-current  
git rev-parse HEAD
```

2. Создать отдельную ветку внедрения

```
git switch -c chore/agent-os-integration
```

Если ветка уже существует:

```
git switch chore/agent-os-integration
```

3. Распаковать Agent OS отдельно от проекта

Рекомендуемый путь:

```
C:\Downloads\agent-os-v0.8
```

Проверка:

```
Test-Path "C:\Downloads\agent-os-v0.8"
```

Просмотр файлов:

```
Get-ChildItem "C:\Downloads\agent-os-v0.8"
```

4. Разрешить выполнение PowerShell-скриптов

Разрешение будет действовать только в текущем окне PowerShell:

```
Set-ExecutionPolicy `
  -Scope Process `
  -ExecutionPolicy Bypass
```

Проверка:

```
Get-ExecutionPolicy -List
```

5. Перейти в распакованный Agent OS

```
Set-Location "C:\Downloads\agent-os-v0.8"
```

Проверить наличие установщика:

```
Test-Path ".\scripts\install-agent-os.ps1"
```

6. Установить Agent OS в AI Advisor

```
.\scripts\install-agent-os.ps1 `
  -RepositoryRoot "C:\AI Advisor"
```

После установки вернуться в проект:

```
Set-Location "C:\AI Advisor"
```

7. Проверить добавленные файлы

```
git status --short
```

Проверить основные каталоги:

```
Test-Path ".\agent-os"  
Test-Path ".\modules\AgentOS"  
Test-Path ".\scripts\agent-os.ps1"  
Test-Path ".\tests\AgentOS.Tests.ps1"  
Test-Path ".\RELEASE-MANIFEST.json"
```

Посмотреть структуру:

```
Get-ChildItem `"  
    ".\agent-os", `"  
    ".\modules\AgentOS", `"  
    ".\scripts", `"  
    ".\tests" `"  
-Recurse
```

8. Проверить PowerShell module manifest

```
Test-ModuleManifest `"  
    ".\modules\AgentOS\AgentOS.psd1"
```

Импортировать модуль:

```
Import-Module `"  
    ".\modules\AgentOS\AgentOS.psd1" `"  
-Force `"  
-Verbose
```

Посмотреть доступные команды:

```
Get-Command -Module AgentOS |  
Sort-Object Name
```

9. Проверить синтаксис PowerShell-файлов

```
Set-Location "C:\AI Advisor"

$errors = @()

$sourcePaths = @(
    ".\modules\AgentOS",
    ".\scripts",
    ".\tests"
)

Get-ChildItem `
    -Path $sourcePaths `
    -Recurse `
    -File `
    -Include *.ps1, *.psm1, *.psd1 |
ForEach-Object {
    $tokens = $null
    $parseErrors = $null

    [System.Management.Automation.Language.Parser]::ParseFile(
        $_.FullName,
        [ref]$tokens,
        [ref]$parseErrors
    ) | Out-Null

    foreach ($parseError in @($parseErrors)) {
        $errors += [pscustomobject]{
            File      = $_.FullName
            Line      = $parseError.Extent.StartLineNumber
            Column    = $parseError.Extent.StartColumnNumber
            Message    = $parseError.Message
        }
    }
}

if ($errors.Count -eq 0) {
    Write-Host "PowerShell syntax: PASSED" -ForegroundColor Green
}
else {
    $errors |
        Sort-Object File, Line, Column |
        Format-Table -AutoSize

    throw "PowerShell syntax validation failed."
}
```

Ожидаемый результат:

```
PowerShell syntax: PASSED
```

10. Проверить наличие Pester

```
Get-Module Pester -ListAvailable |  
Sort-Object Version -Descending |  
Select-Object -First 5 Name, Version, Path
```

Если Pester отсутствует:

```
Install-Module Pester `  
-Scope CurrentUser `  
-Force `  
-SkipPublisherCheck
```

Импортировать Pester:

```
Import-Module Pester -Force
```

Проверить версию:

```
Get-Module Pester |  
Select-Object Name, Version, Path
```

11. Запустить тесты Agent OS

```
Set-Location "C:\AI Advisor"  
  
Invoke-Pester `  
-Path ".\tests" `  
-Output Detailed
```

С сохранением результата:

```
$config = New-PesterConfiguration  
  
$config.Run.Path = ".\tests"  
$config.Output.Verbosity = "Detailed"  
$config.TestResult.Enabled = $true
```

```
$config.TestResult.OutputPath = ".\TestResults\AgentOS.Tests.xml"
$config.TestResult.OutputFormat = "NUnitXml"

Invoke-Pester -Configuration $config
```

12. Инициализировать Agent OS

```
Set-Location "C:\AI Advisor"

.\scripts\agent-os.ps1 init
```

Проверить структуру:

```
Get-ChildItem ".\agent-os" -Force
```

13. Запустить системную диагностику

```
.\scripts\agent-os.ps1 doctor run
```

Проверить внутреннее состояние:

```
.\scripts\agent-os.ps1 system status
```

14. Восстановить состояние после прерванной операции

Сначала убедитесь, что другой процесс Agent OS не работает:

```
Get-Process powershell, pwsh `
-ErrorAction SilentlyContinue |
Select-Object Id, ProcessName, StartTime
```

Обычное восстановление:

```
.\scripts\agent-os.ps1 system recover
```

Принудительное восстановление:

```
.\scripts\agent-os.ps1 system recover -Force
```

`-Force` используйте только после проверки, что активной операции Agent OS действительно нет.

15. Проверить целостность релиза

```
.\scripts\agent-os.ps1 release verify
```

16. Добавить runtime-файлы Agent OS в `.gitignore`

Открыть `.gitignore`:

```
notepad ".\gitignore"
```

Добавить:

```
# Agent OS runtime state
.agent-os/state/
.agent-os/logs/
.agent-os/evidence/
.agent-os/savepoints/
.agent-os/tasks/active/
.agent-os/tasks/completed/
.agent-os/manifests/

# Agent OS test reports
TestResults/
```

Проверить изменения:

```
git diff -- ".gitignore"
```

Не добавляйте правило:

```
.agent-os/
```

Иначе Git будет игнорировать конфигурацию и шаблоны Agent OS.

17. Проверить файлы перед первым commit

```
git status --short
```

Посмотреть только файлы Agent OS:

```
git status --short -- \
  ".agent-os" \
  "modules/AgentOS" \
  "scripts/agent-os.ps1" \
  "scripts/install-agent-os.ps1" \
  "tests/AgentOS.Tests.ps1" \
  "RELEASE-MANIFEST.json" \
  ".gitignore"
```

18. Добавить Agent OS в staging

Добавлять только явные пути:

```
git add -- \
  ".agent-os/config" \
  ".agent-os/templates" \
  "modules/AgentOS" \
  "scripts/agent-os.ps1" \
  "scripts/install-agent-os.ps1" \
  "tests/AgentOS.Tests.ps1" \
  "RELEASE-MANIFEST.json" \
  ".gitignore"
```

Не используйте:

```
git add .
git add -A
```

19. Проверить staged-файлы

```
git diff --cached --name-status
```

Статистика:


```
git diff --cached --stat
```

Полный staged diff:

```
git diff --cached
```

Проверить, что production-код случайно не добавлен:

```
git status --short
```

20. Создать отдельный commit установки

```
git commit `
  -m "chore(agent-os): add deterministic task workflow"
```

Проверить commit:

```
git show `
  --stat `
  --oneline `
  HEAD
```

Настройка Agent OS для проекта AI Advisor

21. Проверить тип проекта

Перейти в корень:

```
Set-Location "C:\AI Advisor"
```

Найти основные project-файлы:

```
Get-ChildItem `
  -Path "." `
  -File `
  -ErrorAction SilentlyContinue |
Where-Object {
  $_.Name -in @(
```

```
        "package.json",
        "pnpm-lock.yaml",
        "package-lock.json",
        "yarn.lock",
        "pyproject.toml",
        "requirements.txt",
        "Cargo.toml",
        "go.mod",
        "pom.xml",
        "build.gradle"
    )
} |
Select-Object Name, FullName
```

22. Проверить Node.js package scripts

Если в проекте есть `package.json`:

```
$packageJsonPath = "C:\AI Advisor\package.json"

if (Test-Path $packageJsonPath) {
    $package = Get-Content $packageJsonPath -Raw |
        ConvertFrom-Json

    $package.scripts |
        Format-List
}
```

23. Проверить package manager

```
$projectRoot = "C:\AI Advisor"

if (Test-Path "$projectRoot\pnpm-lock.yaml") {
    Write-Host "Package manager: pnpm"
}
elseif (Test-Path "$projectRoot\yarn.lock") {
    Write-Host "Package manager: yarn"
}
elseif (Test-Path "$projectRoot\package-lock.json") {
    Write-Host "Package manager: npm"
}
else {
    Write-Host "Node.js lock file not detected"
}
```

24. Проверить доступность pnpm

```
Get-Command pnpm -ErrorAction SilentlyContinue
```

Проверить версию:

```
pnpm --version
```

Если `pnpm` не найден, но установлен Node.js с Corepack:

```
corepack enable  
corepack prepare pnpm@latest --activate
```

После этого:

```
pnpm --version
```

25. Установить зависимости

Для `pnpm`:

```
Set-Location "C:\AI Advisor"  
  
pnpm install --frozen-lockfile
```

Для `npm`:

```
Set-Location "C:\AI Advisor"  
  
npm ci
```

Для `yarn`:

```
Set-Location "C:\AI Advisor"  
  
yarn install --frozen-lockfile
```

Используйте только один package manager, соответствующий lock-файлу проекта.

26. Проверить команды проекта вручную

Примеры для `pnpm`:

```
pnpm lint
```

```
pnpm typecheck
```

```
pnpm test
```

```
pnpm build
```

Запускайте только те команды, которые реально присутствуют в `package.json`.

Проверка конкретного script:

```
$package = Get-Content ".\package.json" -Raw |  
    ConvertFrom-Json  
  
$package.scripts.PSObject.Properties.Name
```

Первая тестовая задача в AI Advisor

27. Проверить незакоммиченное состояние

```
Set-Location "C:\AI Advisor"  
  
git status --short
```

Если есть существующая незакоммиченная работа, Agent OS должен зафиксировать её в baseline и припарковать несвязанные файлы.

28. Найти файлы будущей задачи

Пример поиска по названию компонента:

```
Get-ChildItem `  
    -Path "C:\AI Advisor" `  
    -Recurse `
```

```
-File `
-Filter "*ProductPickerModal*" `
-ErrorAction SilentlyContinue |
Select-Object FullName
```

Для задачи внутри AI Advisor замените `ProductPickerModal` на название реального файла, компонента или модуля.

Например:

```
Get-ChildItem `
-Path "C:\AI Advisor" `
-Recurse `
-File `
-Filter "*Agent*" `
-ErrorAction SilentlyContinue |
Select-Object FullName
```

29. Создать тестовую задачу

Пример:

```
Set-Location "C:\AI Advisor"

.\scripts\agent-os.ps1 task new `
-Title "Validate Agent OS integration" `
-Goal "Run an isolated test change and verify the complete Agent OS
workflow" `
-AllowedScope `
    "tests/**", `
    "docs/agent-os/**" `
-ProtectedScope `
    "src/**", `
    ".env", `
    ".env.*" `
-AutoParkUnrelatedBaseline
```

Для production-задачи замените `AllowedScope` на точные пути изменяемых файлов.

30. Проверить задачу

```
.\scripts\agent-os.ps1 task status
```

```
.\scripts\agent-os.ps1 task manifest
```

```
.\scripts\agent-os.ps1 task validate
```

31. Проверить parked-файлы

```
.\scripts\agent-os.ps1 park list
```

```
.\scripts\agent-os.ps1 park check
```

32. Проверить scope

```
.\scripts\agent-os.ps1 scope check
```

33. Перевести задачу в работу

```
.\scripts\agent-os.ps1 task phase `  
-Phase IN_PROGRESS `  
-Note "Started implementation in C:\AI Advisor"
```

34. Проверить изменения во время работы

```
git status --short
```

```
git diff
```

```
.\scripts\agent-os.ps1 park check
```

```
.\scripts\agent-os.ps1 scope check
```

35. Перевести задачу в READY

```
.\scripts\agent-os.ps1 task phase `
-Phase READY `
-Note "Implementation finished and ready for verification"
```

36. Запустить verification

```
.\scripts\agent-os.ps1 verify run `
-Profile "default"
```

Если настроен отдельный профиль:

```
.\scripts\agent-os.ps1 verify run `
-Profile "ai-advisor"
```

37. Посмотреть evidence

```
Get-ChildItem `
    ".\agent-os\evidence" `
    -Recurse `
    -File |
Sort-Object LastWriteTime -Descending |
Select-Object LastWriteTime, Length, FullName
```

38. Перевести задачу в REVIEWING

```
.\scripts\agent-os.ps1 task phase `
-Phase REVIEWING `
-Note "Verification passed; reviewing final diff"
```

Проверить diff:

```
git diff
```

```
git diff --stat
```

39. Добавить только файлы задачи

Пример:

```
git add -- `
  "tests\AgentOS.Integration.Tests.ps1" `
  "docs\agent-os\integration.md"
```

Используйте реальные пути файлов задачи.

40. Проверить staged scope

```
git diff --cached --name-status
```

```
git diff --cached --stat
```

```
git diff --cached
```

41. Перевести задачу в READY_TO_COMMIT

```
.\scripts\agent-os.ps1 task phase `
  -Phase READY_TO_COMMIT `
  -Note "Diff reviewed and explicit files staged"
```

42. Запустить commit gate

```
.\scripts\agent-os.ps1 commit check
```

Если gate прошёл:

```
git commit `
  -m "test(agent-os): validate AI Advisor integration"
```

43. Получить commit hash

```
$commitHash = git rev-parse HEAD  
  
Write-Host "Commit: $commitHash"
```

44. Завершить задачу

```
.\scripts\agent-os.ps1 task complete ` -CommitHash $commitHash
```

45. Проверить итоговое состояние

```
.\scripts\agent-os.ps1 audit show ` -Last 30
```

```
.\scripts\agent-os.ps1 system status
```

```
.\scripts\agent-os.ps1 doctor run
```

```
git status
```

Ежедневный запуск Agent OS

Чтобы каждый раз не вводить полный путь:

```
Set-Location "C:\AI Advisor"
```

Или:

```
cd "C:\AI Advisor"
```

Запуск диагностики:

```
.\scripts\agent-os.ps1 doctor run
```

Просмотр активной задачи:

```
.\scripts\agent-os.ps1 task status
```

Проверка безопасности:

```
.\scripts\agent-os.ps1 park check  
.\scripts\agent-os.ps1 scope check
```

Проверка транзакций:

```
.\scripts\agent-os.ps1 system status
```

Последние события:

```
.\scripts\agent-os.ps1 audit show -Last 20
```

Удобная переменная проекта

В начале PowerShell-сессии можно определить:

```
$AiAdvisorRoot = "C:\AI Advisor"
```

Переход в проект:

```
Set-Location $AiAdvisorRoot
```

Запуск команд:

```
& "$AiAdvisorRoot\scripts\agent-os.ps1" doctor run
```

```
& "$AiAdvisorRoot\scripts\agent-os.ps1" task status
```

```
& "$AiAdvisorRoot\scripts\agent-os.ps1" system status
```

Удобная PowerShell-функция

Добавьте в текущую сессию:

```
function Invoke-AiAdvisorAgentOs {  
    param(  
        [Parameter(ValueFromRemainingArguments)]  
        [string[]]$Arguments  
    )  
  
    $projectRoot = "C:\AI Advisor"  
    $cliPath = Join-Path $projectRoot "scripts\agent-os.ps1"  
  
    if (-not (Test-Path -LiteralPath $cliPath)) {  
        throw "Agent OS CLI not found: $cliPath"  
    }  
  
    Push-Location $projectRoot  
  
    try {  
        & $cliPath @Arguments  
    }  
    finally {  
        Pop-Location  
    }  
}
```

Примеры:

```
Invoke-AiAdvisorAgentOs doctor run
```

```
Invoke-AiAdvisorAgentOs task status
```

```
Invoke-AiAdvisorAgentOs system status
```

```
Invoke-AiAdvisorAgentOs audit show -Last 20
```

Минимальная последовательность внедрения

```
Set-Location "C:\AI Advisor"

git status
git switch -c chore/agent-os-integration

Set-ExecutionPolicy `
  -Scope Process `
  -ExecutionPolicy Bypass

Set-Location "C:\Downloads\agent-os-v0.8"

.\scripts\install-agent-os.ps1 `
  -RepositoryRoot "C:\AI Advisor"

Set-Location "C:\AI Advisor"

Test-ModuleManifest ".\modules\AgentOS\AgentOS.psd1"

Import-Module `
  ".\modules\AgentOS\AgentOS.psd1" `
  -Force

Invoke-Pester `
  -Path ".\tests" `
  -Output Detailed

.\scripts\agent-os.ps1 init

.\scripts\agent-os.ps1 release verify

.\scripts\agent-os.ps1 doctor run

git status --short
```

После успешных проверок:

```
git add -- `
  ".agent-os/config" `
  ".agent-os/templates" `
  "modules/AgentOS" `
  "scripts/agent-os.ps1" `
  "scripts/install-agent-os.ps1" `
  "tests/AgentOS.Tests.ps1" `
  "RELEASE-MANIFEST.json" `
```

```
    ".gitignore"

git diff --cached

git commit `
  -m "chore(agent-os): add deterministic task workflow"
```